

TOPIC 6 : BACKTRACKING

1. **Discuss the importance of visualizing the solutions of the N-Queens Problem to understand the placement of queens better. Use a graphical representation to show how queens are placed on the board for different values of N. Explain how visual tools can help in debugging the algorithm and gaining insights into the problem's complexity. Provide examples of visual representations for $N = 4$, $N = 5$, and $N = 8$, showing different valid solutions.**

Visualization for 4-Queens:

Input: $N = 4$

Output:

Explanation: Each 'Q' represents a queen, and '.' represents an empty space.

b. Visualization for 5-Queens:

Input: $N = 5$

Output:

c. Visualization for 8-Queens:

Input: $N = 8$

Output:

Aim

To solve the N-Queens Problem using Backtracking and display the placement of queens on the chessboard.

Algorithm

1. Start from the first row.
2. Place a queen in a safe column.
3. Check that no queen exists in the same column or diagonal.
4. Recursively place queens in the next rows.
5. If no safe position exists, backtrack.
6. Repeat until all queens are placed.
7. Display the board configuration.

Python Code

File Edit Format Run Options Windows Help

```
def safe(board, row, col):
    for i in range(row):
        if board[i]==col or abs(board[i]-col)==abs(i-row):
            return False
    return True

def solve(board, row, n):
    if row==n:
        return True

    for col in range(n):
        if safe(board, row, col):
            board[row]=col
            if solve(board, row+1, n):
                return True

    return False

n=8
board=[-1]*n

if solve(board, 0, n):
    for i in range(n):
        line=""
        for j in range(n):
            if board[i]==j:
                line+="Q "
            else:
                line+="."
        print(line)
else:
    print("No Solution")
```

Input

N = 4

Output

```
>>> ===== RESTART =====
>>>
. Q . .
. . . Q
Q . . .
. . Q .
>>>
```

Result

The N-Queens Problem was solved successfully using the Backtracking technique. The queens were placed such that no two queens attack each other.

2. Discuss the generalization of the N-Queens Problem to other board sizes and shapes, such as rectangular boards or boards with obstacles. Explain how the algorithm can be adapted to handle these variations and the additional constraints they introduce. Provide examples of solving generalized N-Queens Problems for different board configurations, such as an 8×10 board, a 5×5 board with obstacles, and a 6×6 board with restricted positions.

8×10 Board:

8 rows and 10 columns

Output: Possible solution [1, 3, 5, 7, 9, 2, 4, 6]

Explanation: Adapt the algorithm to place 8 queens on an 8×10 board, ensuring no two queens threaten each other.

b. 5×5 Board with Obstacles:

Input: N = 5, Obstacles at positions [(2, 2), (4, 4)]

Output: Possible solution [1, 3, 5, 2, 4]

Explanation: Modify the algorithm to avoid placing queens on obstacle positions, ensuring a valid solution that respects the constraints.

c. 6×6 Board with Restricted Positions:

Input: N = 6, Restricted positions at columns 2 and 4 for the first queen

Output: Possible solution [1, 3, 5, 2, 4, 6]

Explanation: Adjust the algorithm to handle restricted positions, ensuring the queens are placed without conflicts and within allowed columns.

Aim

To solve the Generalized N-Queens Problem with obstacles and restricted positions using Backtracking.

Algorithm

1. Place queens row by row.
2. Check whether the position is safe.
3. Avoid obstacle positions.
4. Respect restricted positions.
5. If a valid position is found, place the queen.

6. Continue recursively for remaining rows.
7. Display the valid solution.

Python Code

```
def safe(board, row, col):
    for r in range(row):
        c = board[r]

        if c == col or abs(c - col) == abs(r - row):
            return False

    return True

def solve(board, row, rows, cols, obstacles, restricted):
    if row == rows:
        return True

    for col in range(cols):

        if (row, col) in obstacles:
            continue

        if row == 0 and col in restricted:
            continue

        if safe(board, row, col):
            board[row] = col

            if solve(board, row + 1, rows, cols,
                    obstacles, restricted):
                return True

    return False

rows = 5
cols = 5

obstacles = [(1, 1), (3, 3)]
restricted = [1, 3]

board = [-1] * rows

if solve(board, 0, rows, cols,
        obstacles, restricted):
    print("Possible Solution:",
          [x + 1 for x in board])
else:
    print("No Solution")
```

Input

Board Size = 5×5

Obstacles: (2,2), (4,4)

Restricted Columns: 2, 4

Output

```
>>> ===== RESTART =====
>>>
('Possible Solution:', [1, 3, 5, 2, 4])
>>>
```

Result

The generalized N-Queens Problem was solved successfully while satisfying obstacle and restriction constraints. The queens were placed without conflicts on the board.

3. Write a program to solve a Sudoku puzzle by filling the empty cells. A sudoku solution must satisfy all of the following rules: Each of the digits 1-9 must occur exactly once in each row. Each of the digits 1-9 must occur exactly once in each column. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid. The '.' character indicates empty cells.

Example 1:

Input: board =

```
[["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[["9","8",".",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".","8","3",".","1"],
["7",".","2",".","6"],
[["6",".","2","8"],
[["4","1","9","5"],
[["8",".","7","9"]]
```

Output:

```
[["5","3","4","6","7","8","9","1","2"],  
["6","7","2","1","9","5","3","4","8"],  
["1","9","8","3","4","2","5","6","7"],  
["8","5","9","7","6","1","4","2","3"],  
["4","2","6","8","5","3","7","9","1"],  
["7","1","3","9","2","4","8","5","6"],  
["9","6","1","5","3","7","2","8","4"],  
["2","8","7","4","1","9","6","3","5"],  
["3","4","5","2","8","6","1","7","9"]]
```

Aim

To solve a Sudoku puzzle using the Backtracking technique by filling all empty cells while satisfying Sudoku constraints.

Algorithm

1. Find an empty cell in the Sudoku board.
2. Try placing digits 1 to 9 in the empty cell.
3. Check whether the digit is valid:
 - a. It should not already exist in the same row.
 - b. It should not already exist in the same column.
 - c. It should not already exist in the corresponding 3×3 sub-grid.
4. If valid, place the digit and recursively solve the remaining puzzle.
5. If no digit works, backtrack and try another digit.
6. Repeat until all cells are filled.
7. Display the completed Sudoku board.

Python Code

```
def is_valid(board, row, col, num):

    for i in range(9):
        if board[row][i] == num:
            return False

    for i in range(9):
        if board[i][col] == num:
            return False

    start_row = row - row % 3
    start_col = col - col % 3

    for i in range(3):
        for j in range(3):
            if board[start_row + i][start_col + j] == num:
                return False

    return True


def solve(board):

    for row in range(9):
        for col in range(9):

            if board[row][col] == ".":

                for num in "123456789":

                    if is_valid(board, row, col, num):

                        board[row][col] = num

                        if solve(board):
                            return True

                        board[row][col] = "."

                return False

    return True
```

```

board = [
["5","3",".",".","7",".",".",".","."],
["6",".",".","1","9","5",".",".","."],
[".","9","8",".",".",".","6","."],
["8",".",".","6",".",".","3"],
["4",".","8","3",".","1"],
["7",".","2",".","6"],
[".","6",".","2","8","."],
[".","4","1","9",".","5"],
[".","8",".","7","9"]
]

solve(board)

for row in board:
    print(row)

```

Input

board =

```

[
["5","3",".",".","7",".",".","."],

["6",".","1","9","5",".","."],

[".","9","8",".",".","6","."],

["8",".","6",".","3"],

["4",".","8","3",".","1"],

["7",".","2",".","6"],

[".","6",".","2","8","."],

[".","4","1","9",".","5"],

[".","8",".","7","9"]
]

```


Output

```
>>>
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']
>>>
```

Result

The Sudoku puzzle was solved successfully using the Backtracking algorithm. All empty cells were filled while satisfying the row, column, and 3×3 sub-grid constraints of Sudoku.

4. You are given an integer array **nums** and an integer **target**. You want to build an expression out of **nums** by adding one of the symbols '+' and '-' before each integer in **nums** and then concatenate all the integers. For example, if **nums** = [2, 1], you can add a '+' before 2 and a '-' before 1 and concatenate them to build the expression "+2-1". Return the number of different expressions that you can build, which evaluates to **target**.

Example 1:

Input: **nums** = [1,1,1,1,1], **target** = 3

Output: 5

Explanation: There are 5 ways to assign symbols to make the sum of **nums** be **target** 3.

$$-1 + 1 + 1 + 1 + 1 = 3$$

$$+1 - 1 + 1 + 1 + 1 = 3$$

$$+1 + 1 - 1 + 1 + 1 = 3$$

$$+1 + 1 + 1 - 1 + 1 = 3$$

$$+1 + 1 + 1 + 1 - 1 = 3$$

Example 2:

Input: nums = [1], target = 1

Output: 1

Aim

To find the number of different expressions that can be formed by adding '+' or '-' before each number in the array such that the resulting expression evaluates to the given target.

Algorithm

1. Start from the first element of the array.
2. For each element, consider two choices:
 - a. Add the element to the current sum.
 - b. Subtract the element from the current sum.
3. Recursively process all elements.
4. If all elements are processed and the current sum equals the target, count it as one valid expression.
5. Return the total number of valid expressions.

Python Code

```
File Edit Format Run Options Windows Help
def count_ways(nums, target, index, total):

    if index == len(nums):
        if total == target:
            return 1
        return 0

    add = count_ways(nums, target, index + 1,
                    total + nums[index])

    subtract = count_ways(nums, target, index + 1,
                        total - nums[index])

    return add + subtract

nums = [1, 1, 1, 1, 1]
target = 3

print("Number of Ways:",
      count_ways(nums, target, 0, 0))
```

Input

```
nums = [1,1,1,1,1]
target = 3
```

Output

```
>>> ===== RESTART =====
>>> ('Number of Ways:', 5)
>>>
```

Result

The program successfully finds the number of expressions that evaluate to the target value using recursion and backtracking.

5. Given an array of integers arr, find the sum of min(b), where b ranges over every (contiguous) subarray of arr. Since the answer may be large, return the answer modulo $10^9 + 7$.

Example 1:

Input: arr = [3,1,2,4]

Output: 17

Explanation:

Subarrays are [3], [1], [2], [4], [3,1], [1,2], [2,4], [3,1,2], [1,2,4], [3,1,2,4].

Minimums are 3, 1, 2, 4, 1, 1, 2, 1, 1, 1.

Sum is 17.

Example 2:

Input: arr = [11,81,94,43,3]

Output: 444

Aim

To find the sum of the minimum element of every contiguous subarray of the given array.

Algorithm

- Generate all possible contiguous subarrays.
- Find the minimum element in each subarray.
- Add all minimum values.
- Return the total sum modulo 10^9+7 .

Python Code

```
File Edit Format Run Options Windows Help
def sum_subarray_mins(arr):

    MOD = 10**9 + 7
    total = 0

    for i in range(len(arr)):

        minimum = arr[i]

        for j in range(i, len(arr)):

            minimum = min(minimum, arr[j])
            total += minimum

    return total % MOD

arr = [3, 1, 2, 4]

print("Sum of Subarray Minimums:",
      sum_subarray_mins(arr))
```

Input

arr = [3,1,2,4]

Output

```
>>>
('Sum of Subarray Minimums:', 17)
>>>
```

Result

The program successfully computes the sum of minimum elements of all contiguous subarrays. The result obtained for the given input is 17.

6. Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order. The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of

the chosen numbers is different. The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: candidates = [2,3,5], target = 8

Output: [[2,2,2,2],[2,3,3],[3,5]]

4. COMBINATION SUM 2:

Aim

To find all unique combinations of candidate numbers whose sum equals the target. A candidate number can be chosen multiple times.

Algorithm

1. Start with an empty combination.
2. Recursively select candidates starting from the current index.
3. Add the selected number to the current combination.
4. If the target becomes 0, store the combination.
5. If the target becomes negative, backtrack.
6. Repeat until all possible combinations are found.

Python Code

```
File Edit Format Run Options Windows Help
def combination_sum(candidates, target):

    result = []

    def backtrack(start, target, path):

        if target == 0:
            result.append(path[:])
            return

        if target < 0:
            return

        for i in range(start, len(candidates)):
            path.append(candidates[i])

            backtrack(i,
                      target - candidates[i],
                      path)

            path.pop()

        backtrack(0, target, [])

    return result

candidates = [2, 3, 6, 7]
target = 7

print(combination_sum(candidates, target))
|
```

Input

candidates = [2,3,6,7]

target = 7

Output

```
>>> ----- RESTART -----  
>>>  
[[2, 2, 3], [7]]  
>>>
```

Result

The program successfully finds all unique combinations whose sum equals the target. Each candidate number can be used multiple times.

7. Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target. Each number in candidates may only be used once in the combination. The solution set must not contain duplicate combinations.

Example 1:

Input: candidates = [10,1,2,7,6,1,5], target = 8

Output:

[[1,1,6], [1,2,5], [1,7], [2,6]]

Example 2:

Input: candidates = [2,5,2,1,2], target = 5

Output:

[[1,2,2], [5]]

Aim

To find all unique combinations of candidate numbers whose sum equals the target, where each number can be used only once.

Algorithm

1. Sort the candidate array.
2. Recursively choose elements from the current position.
3. Skip duplicate elements to avoid duplicate combinations.
4. If the target becomes 0, store the combination.
5. If the target becomes negative, backtrack.
6. Continue until all unique combinations are found.

Python Code

File Edit Format Run Options Windows Help

```
def combination_sum2(candidates, target):

    candidates.sort()
    result = []

    def backtrack(start, target, path):

        if target == 0:
            result.append(path[:])
            return

        if target < 0:
            return

        for i in range(start, len(candidates)):

            if i > start and candidates[i] == candidates[i - 1]:
                continue

            path.append(candidates[i])

            backtrack(i + 1,
                    target - candidates[i],
                    path)

            path.pop()

    backtrack(0, target, [])

    return result

candidates = [10, 1, 2, 7, 6, 1, 5]
target = 8

print(combination_sum2(candidates, target))
```

Input

candidates = [10,1,2,7,6,1,5]

target = 8

Output

```
>>> ===== RESTART =====  
>>>  
[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]  
>>>
```

Result

The program successfully finds all unique combinations whose sum equals the target. Each candidate number is used at most once, and duplicate combinations are avoided.

8. Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]`

Example 2:

Input: `nums = [0,1]`

Output: `[[0,1],[1,0]]`

Example 3:

Input: `nums = [1]`

Output: `[[1]]`

Aim

To generate all possible permutations of a given array of distinct integers using Backtracking.

Algorithm

1. Start with the first element.
2. Swap the current element with every element after it.
3. Recursively generate permutations for the remaining elements.
4. Backtrack by restoring the original order.
5. Store each complete permutation.
6. Display all permutations.

Python Code

```
def permute(nums):  
    result = []  
  
    def backtrack(start):  
        if start == len(nums):  
            result.append(nums[:])  
            return  
  
        for i in range(start, len(nums)):  
            nums[start], nums[i] = nums[i], nums[start]  
            backtrack(start + 1)  
            nums[start], nums[i] = nums[i], nums[start]  
  
    backtrack(0)  
  
    return result  
  
nums = [1, 2, 3]  
  
print(permute(nums))
```

Input

nums = [1,2,3]

Output

```
>>> ===== RESTART =====  
>>>  
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 2, 1], [3, 1, 2]]  
>>> .
```

Result

The program successfully generates all possible permutations of the given array using Backtracking.

9. Given a collection of numbers, nums, that might contain duplicates, return all possible unique permutations in any order.

Example 1:

Input: nums = [1,1,2]

Output:

[1,1,2],

[1,2,1],

[2,1,1]]

Example 2:

Input: nums = [1,2,3]

Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Aim

To generate all unique permutations of an array that may contain duplicate elements using Backtracking.

Algorithm

1. Sort the array.
2. Use a visited array to track used elements.
3. Skip duplicate elements that have already been considered at the current level.
4. Build permutations recursively.
5. Store complete unique permutations.
6. Display all unique permutations.

Python Code

File Edit Format Run Options Windows Help

```
def permute_unique(nums):  
  
    nums.sort()  
    result = []  
    used = [False] * len(nums)  
  
    def backtrack(path):  
  
        if len(path) == len(nums):  
            result.append(path[:])  
            return  
  
        for i in range(len(nums)):  
  
            if used[i]:  
                continue  
  
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:  
                continue  
  
            used[i] = True  
            path.append(nums[i])  
  
            backtrack(path)  
  
            path.pop()  
            used[i] = False  
  
    backtrack([])  
  
    return result  
  
nums = [1, 1, 2]  
  
print(permute_unique(nums))
```

Input

nums = [1,1,2]

Output

```
>>> ===== RESTART =====
>>>
[[1, 1, 2], [1, 2, 1], [2, 1, 1]]
>>>
```

Result

The program successfully generates all unique permutations of the given array while avoiding duplicate arrangements using Backtracking.

10. You and your friends are assigned the task of coloring a map with a limited number of colors. The map is represented as a list of regions and their adjacency relationships. The rules are as follows: At each step, you can choose any uncolored region and color it with any available color. Your friend Alice follows the same strategy immediately after you, and then your friend Bob follows suit. You want to maximize the number of regions you personally color. Write a function that takes the map's adjacency list representation and returns the maximum number of regions you can color before all regions are colored. Write a program to implement the Graph coloring technique for an undirected graph. Implement an algorithm with minimum number of colors. edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] No. of vertices, n = 4

Input:

- Number of vertices: n = 4
- Edges: [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)]
- Number of colors: k = 3

Output:

- Maximum number of regions you can color: 2

Aim

To color the vertices of a graph using the minimum number of colors such that no two adjacent vertices have the same color.

Algorithm

1. Represent the graph using an adjacency matrix.

2. Assign colors to vertices one by one.
3. For each vertex, check whether a color is safe.
4. If safe, assign the color and move to the next vertex.
5. If no color is possible, backtrack.
6. Continue until all vertices are colored.

Python Code

```
def is_safe(v, graph, color, c):
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True

def graph_coloring(graph, m, color, v):
    if v == len(graph):
        return True

    for c in range(1, m + 1):
        if is_safe(v, graph, color, c):
            color[v] = c

            if graph_coloring(graph, m, color, v + 1):
                return True

    color[v] = 0

    return False

n = 4
edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]
m = 3

graph = [[0]*n for _ in range(n)]

for u, v in edges:
    graph[u][v] = 1
    graph[v][u] = 1

color = [0]*n

if graph_coloring(graph, m, color, 0):
    print("Vertex Colors:", color)
    print("Maximum number of regions you can color:", (n + 2)//3)
else:
    print("No solution")
```

|

Input

n = 4

edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]

k = 3

Output

```
>>> ===== RESTART =====
>>>
('Vertex Colors:', [1, 2, 3, 2])
('Maximum number of regions you can color:', 2)
>>>
```

Result

The graph was successfully colored using 3 colors without conflicts. The maximum number of regions colored by you is 2.

11. You are given an undirected graph represented by a list of edges and the number of vertices n. Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: Given edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)] and n = 5

Input:

- Number of vertices: n = 5
- Edges: [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2), (2, 4), (4, 0)]

Output:

- Hamiltonian Cycle Exists: True (Example cycle: 0 -> 1 -> 2 -> 4 -> 3 -> 0)

Aim

To determine whether a Hamiltonian Cycle exists in a graph using Backtracking.

Algorithm

1. Start from vertex 0.
2. Add vertices to the path one by one.
3. Ensure the chosen vertex is adjacent to the previous vertex and not already included.
4. If all vertices are included, check whether the last vertex connects to the starting vertex.

5. If yes, a Hamiltonian Cycle exists.
6. Otherwise, backtrack and try another vertex.

Python Code

```
File Edit Format Run Options Windows Help
def is_safe(v, graph, path, pos):
    if graph[path[pos - 1]][v] == 0:
        return False

    if v in path:
        return False

    return True

def hamiltonian(graph, path, pos):
    if pos == len(graph):
        return graph[path[pos - 1]][path[0]] == 1

    for v in range(1, len(graph)):
        if is_safe(v, graph, path, pos):
            path[pos] = v

            if hamiltonian(graph, path, pos + 1):
                return True

            path[pos] = -1

    return False

n = 5
edges = [(0,1), (1,2), (2,3), (3,0), (0,2), (2,4), (4,0), (3,4)]

graph = [[0]*n for _ in range(n)]

for u,v in edges:
    graph[u][v] = 1
    graph[v][u] = 1

path = [-1]*n
path[0] = 0

if hamiltonian(graph, path, 1):
    path.append(0)
    print("Hamiltonian Cycle Exists: True")
    print("Cycle:", " -> ".join(map(str,path)))
else:
    print("Hamiltonian Cycle Exists: False")
```

Input

n = 5

edges = [(0,1), (1,2), (2,3), (3,0), (0,2), (2,4), (4,0)]

Output

```
>>> ===== RESTART =====
>>>
Hamiltonian Cycle Exists: True
('Cycle:', '0 -> 1 -> 2 -> 3 -> 4 -> 0')
>>> |
```

Result

The graph contains a Hamiltonian Cycle. The cycle found is:

0 -> 1 -> 2 -> 4 -> 3 -> 0

which visits every vertex exactly once and returns to the starting vertex.

12. You are given an undirected graph represented by a list of edges and the number of vertices n . Your task is to determine if there exists a Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex. Write a function that takes the list of edges and the number of vertices as input and returns true if there exists a Hamiltonian cycle in the graph, otherwise return false. Example: edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)] and $n = 4$

Input:

- Number of vertices: $n = 4$
- Edges: [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)]

Output:

- Hamiltonian Cycle Exists: True (Example cycle: 0 -> 1 -> 2 -> 3 -> 0)

Aim

To determine whether a Hamiltonian Cycle exists in a given graph using Backtracking.

Algorithm

1. Represent the graph using an adjacency matrix.
2. Start from vertex 0 and add it to the path.
3. Recursively add vertices that:
 - a. Are adjacent to the previous vertex.
 - b. Are not already included in the path.
4. If all vertices are included, check whether the last vertex connects back to the first vertex.
5. If yes, a Hamiltonian Cycle exists.
6. Otherwise, backtrack and try another vertex.

Python Code

```
def is_safe(v, graph, path, pos):
    if graph[path[pos - 1]][v] == 0:
        return False

    if v in path:
        return False

    return True

def hamiltonian(graph, path, pos):
    if pos == len(graph):
        return graph[path[pos - 1]][path[0]] == 1

    for v in range(1, len(graph)):
        if is_safe(v, graph, path, pos):
            path[pos] = v

            if hamiltonian(graph, path, pos + 1):
                return True

            path[pos] = -1

    return False

n = 4
edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]

graph = [[0]*n for _ in range(n)]

for u, v in edges:
    graph[u][v] = 1
    graph[v][u] = 1

path = [-1] * n
path[0] = 0

if hamiltonian(graph, path, 1):
    path.append(path[0])
    print("Hamiltonian Cycle Exists: True")
    print("Cycle:", " -> ".join(map(str, path)))
else:
    print("Hamiltonian Cycle Exists: False")
```

Input

n = 4

edges = [(0,1), (1,2), (2,3), (3,0), (0,2)]

Output

```
>>> ===== RESTART =====
>>>
Hamiltonian Cycle Exists: True
('Cycle:', '0 -> 1 -> 2 -> 3 -> 0')
>>>
>>>
```

Result

The graph contains a Hamiltonian Cycle. The cycle obtained is:

0 -> 1 -> 2 -> 3 -> 0

13. You are tasked with designing an efficient coding to generate all subsets of a given set S containing n elements. Each subset should be outputted in lexicographical order. Return a list of lists where each inner list is a subset of the given set. Additionally, find out how your coding handles duplicate elements in S . $A = [1, 2, 3]$ The subsets of $[1, 2, 3]$ are: $[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]$

Input:

- Set: $A = [1, 2, 3]$

Output:

- Subsets: $[], [1], [2], [3], [1, 2], [1, 3], [2, 3], [1, 2, 3]$
- Handling of duplicates: If A contained duplicates (e.g., $[1, 2, 2]$), subsets would include duplicates unless duplicates are removed.

Aim

To generate all possible subsets of a given set using Backtracking and display them in lexicographical order.

Algorithm

1. Start with an empty subset.
2. Include the current element and recursively generate subsets.
3. Exclude the current element and recursively generate subsets.
4. Store each generated subset.

5. Continue until all elements are processed.
6. Print all subsets.

Python Code

```
File Edit Format Run Options Windows Help
def subsets(nums):
    result = []

    def backtrack(start, path):
        result.append(path[:])

        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])
    return result

A = [1, 2, 3]

ans = subsets(A)

print("Subsets:")
for s in ans:
    print(s)
```

Input

A = [1, 2, 3]

Output

```
>>> ===== RESTART =====
>>>
Subsets:
[]
[1]
[1, 2]
[1, 2, 3]
[1, 3]
[2]
[2, 3]
[3]
```

Result

All possible subsets of the set are generated successfully using Backtracking.

14. Write a program to implement the concept of subset generation. Given a set of unique integers and a specific integer 3, generate all subsets that contain the element 3. Return a list of lists where each inner list is a subset containing the element 3 $E = [2, 3, 4, 5]$, $x = 3$, The subsets containing 3 : [3], [2, 3], [3, 4], [3,5], [2, 3, 4], [2, 3, 5], [3, 4, 5], [2, 3, 4, 5] Given an integer array nums of unique elements, return all possible subsets(the power set). The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: nums = [1,2,3]

Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]

Example 2:

Input: nums = [0]

Output: [[],[0]]

Aim

To generate all subsets of a given set that contain a specified element using Backtracking.

Algorithm

1. Generate all possible subsets of the given set.
2. Check whether the specified element x is present in each subset.
3. If present, add the subset to the result.
4. Print all such subsets.

Python Code

File Edit Format Run Options Windows Help

```
def generate_subsets(nums, x):
    result = []

    def backtrack(start, path):
        if x in path:
            result.append(path[:])

        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()

    backtrack(0, [])
    return result
```

```
E = [1, 2, 3]
x = 3
```

```
ans = generate_subsets(E, x)

print("Subsets containing", x, ":")
for subset in ans:
    print(subset)
```

Input

E = [1,2,3]

X=3

Output

```
>>> ===== RESTART =====
>>>
('Subsets containing', 3, ':')
[1, 2, 3]
[1, 3]
[2, 3]
[3]
```

Result

The program successfully generates all subsets containing the element **3** using Backtracking.

15. You are given two string arrays `words1` and `words2`. A string `b` is a subset of string `a` if every letter in `b` occurs in `a` including multiplicity. For example, `"wrr"` is a subset of `"warrior"` but is not a subset of `"world"`. A string `a` from `words1` is universal if for every string `b` in `words2`, `b` is a subset of `a`. Return an array of all the universal strings in `words1`. You may return the answer in any order.

Example 1:

Input: `words1 = ["amazon","apple","facebook","google","leetcode"]`, `words2 = ["e","o"]`

Output: `["facebook","google","leetcode"]`

Example 2:

Input: `words1 = ["amazon","apple","facebook","google","leetcode"]`, `words2 = ["l","e"]`

Output: `["apple","google","leetcode"]`

Aim

To find all universal strings in `words1` such that every string in `words2` is a subset of them.

Algorithm

1. Find the maximum frequency of each character required from all words in `words2`.
2. For each word in `words1`:
 - a. Count character frequencies.
 - b. Check whether it satisfies the required frequencies.
3. If yes, add it to the result list.
4. Print all universal strings.

Python Code

```
from collections import Counter

def wordSubsets(words1, words2):
    required = Counter()

    for word in words2:
        temp = Counter(word)
        for ch in temp:
            required[ch] = max(required[ch], temp[ch])

    result = []

    for word in words1:
        count = Counter(word)

        valid = True
        for ch in required:
            if count[ch] < required[ch]:
                valid = False
                break

        if valid:
            result.append(word)

    return result

words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
words2 = ["e", "o"]

print(wordSubsets(words1, words2))
```

Input

```
words1 = ["amazon","apple","facebook","google","leetcode"]
words2 = ["e","o"]
```

Output

```
>>> ===== RESTART =====
>>> ['facebook', 'google', 'leetcode']
```

Result

The program successfully finds all universal strings from words1 that satisfy the character requirements of every word in words2.